

DOCUMENTAÇÃO TÉCNICA

# Benevita

Plataforma de gestão de cuidado domiciliar. Arquitetura, tecnologias, perfis de uso e investimento em infraestrutura.

---

Preparado por Iago

Documento de engenharia de software

# 1. Visão geral

O Benevita é um aplicativo mobile para empresas de cuidado domiciliar. Ele reúne num só lugar o que costuma viver espalhado em caderno e no WhatsApp, e conecta três públicos: a empresa que gere a operação, o cuidador que atende na casa do paciente e a família que acompanha o cuidado.

O app roda em iPhone e Android a partir de uma mesma base de código, funciona mesmo sem internet e sincroniza os registros quando a conexão volta. Toda a informação é isolada por empresa, e cada perfil enxerga apenas o que lhe cabe.

- **Empresa (admin):** gere pacientes, equipe, escalas e financeiro num painel único.
- **Cuidador:** inicia o plantão e registra o cuidado direto do celular, com ou sem sinal.
- **Família:** acompanha o dia do paciente, os sinais vitais e o histórico em tempo real.

## 2. Arquitetura e tecnologias

O Benevita é um aplicativo nativo multiplataforma construído com React Native e Expo, escrito em TypeScript. O backend é o Firebase, acessado direto pelo aplicativo através do SDK oficial. Não há servidor próprio a manter: a segurança e o acesso aos dados são garantidos pelas regras do Firebase, e não pelo aplicativo, já que ele roda no aparelho do usuário.

### 2.1 Camada do aplicativo

| Tecnologia                      | Como é usada                                                                              |
|---------------------------------|-------------------------------------------------------------------------------------------|
| React Native 0.81 e Expo SDK 54 | Base do app, uma única base de código gera as versões de iPhone e Android.                |
| TypeScript (modo estrito)       | Tipagem de todo o código, o que reduz erros antes mesmo de rodar.                         |
| React Navigation v7             | Navegação entre telas e as abas inferiores, montadas de forma diferente para cada perfil. |
| Zustand                         | Guarda o estado da sessão e do usuário logado, disponível para todas as telas.            |
| React Hook Form e Zod           | Montam e validam os formulários de registro, garantindo que só dado válido é salvo.       |
| AsyncStorage                    | Mantém o login entre sessões e guarda a fila de registros feitos offline.                 |
| date-fns                        | Formatação e cálculo de datas e horários.                                                 |
| expo-linear-gradient            | Gradientes e acabamento visual da interface.                                              |

### 2.2 Recursos do aparelho e mídia

| Tecnologia    | Como é usada                                                                           |
|---------------|----------------------------------------------------------------------------------------|
| expo-location | Registra a localização no início do plantão, para comprovar a presença no atendimento. |

|                                                      |                                                                                       |
|------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>expo-image-picker e expo-image-manipulator</b>    | Captura a foto do registro, comprime e converte para um formato leve antes de salvar. |
| <b>@react-native-community/datetimepicker</b>        | Seletores nativos de data e hora, no formato de roda, seguindo o padrão do iOS.       |
| <b>react-native-chart-kit e react-native-svg</b>     | Desenham o gráfico de sinais vitais que a família acompanha.                          |
| <b>react-native-calendars</b>                        | Calendário usado na montagem das escalas.                                             |
| <b>react-native-html-to-pdf e react-native-share</b> | Geram e compartilham relatórios em PDF a partir dos registros.                        |

## 2.3 Backend e dados (Firebase)

| Tecnologia                      | Como é usada                                                                                                              |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Firebase Authentication</b>  | Login por email e senha, troca de senha e persistência da sessão no aparelho.                                             |
| <b>Cloud Firestore</b>          | Banco em tempo real. Guarda pacientes, registros, escalas, plantões, financeiro e auditoria, tudo isolado por empresa.    |
| <b>Firestore Security Rules</b> | Controlam quem acessa o quê. Negam por padrão, isolam por empresa e por perfil, e tornam os registros clínicos imutáveis. |
| <b>Fila offline (própria)</b>   | Registros feitos sem internet entram numa fila local e sincronizam sozinhos quando a conexão volta.                       |
| <b>EAS Build</b>                | Serviço da Expo que compila o app para iPhone e Android na nuvem.                                                         |

Uma decisão importante de arquitetura: as fotos dos registros são guardadas dentro do próprio documento no Firestore, em formato de texto comprimido, e não no Firebase Storage. Isso simplifica o app hoje, mas pesa mais no investimento de infraestrutura conforme o volume cresce, como explicado na seção de investimento.

### 3. Perfis de uso e telas

O Benevita tem três perfis, cada um com o seu conjunto de telas e o seu fluxo. Ao todo são 36 telas, além de telas compartilhadas de perfil, ajuda, troca de senha e consentimento de dados.

#### 3.1 Empresa (Admin)

É o perfil de gestão. Enxerga a operação inteira e configura tudo: equipe, pacientes, escalas e financeiro.

##### Telas:

- Setup da empresa, primeiro acesso que cria a empresa.
- Dashboard, painel com pacientes ativos, profissionais, plantões em andamento, registros e intercorrências do dia, mais um resumo do financeiro.
- Lista de pacientes e detalhe do paciente, com histórico e exportação de relatório.
- Cadastro de paciente, criando o registro e vinculando à família.
- Equipe, detalhe do cuidador e cadastro de cuidador, para montar e gerir o time.
- Escalas, definindo qual cuidador cuida de qual paciente em cada dia.
- Financeiro, com receitas, despesas, saldo do mês e exportação em PDF.
- Convite e vínculo de família, e edição dos dados da empresa.

**Fluxo de uso:** o admin cria a empresa, convida a família, cadastra o paciente e o vincula, monta a equipe de cuidadores, define as escalas e passa a acompanhar tudo pelo dashboard e pelo financeiro.

#### 3.2 Cuidador

É o perfil de campo. Trabalha na casa do paciente e registra o cuidado direto do celular.

##### Telas:

- Login.
- Início, com a lista de pacientes e o card da escala do dia.
- Check in do plantão, que só libera o paciente escalado para hoje e registra a localização.
- Detalhe do paciente, com o guia fixo de informações e o histórico recente.
- Registro rápido e as telas de sinais vitais, alimentação, medicação, atividade, intercorrência e foto.
- Evolução e passagem de plantão, com o checkout.
- Histórico de plantões e exportação de relatório.

**Fluxo de uso:** o cuidador faz login, vê a escala do dia, dá check in no paciente daquele dia, registra os cuidados ao longo do plantão, mesmo sem sinal, escreve a evolução e faz o checkout ao final.

#### 3.3 Família

É o perfil de acompanhamento. Cadastra o paciente e acompanha o cuidado sem interferir na operação.

##### Telas:

- Tela de espera, quando ainda não há paciente vinculado.
- Onboarding, o passo a passo que cadastra o paciente e os dados clínicos.
- Tela do paciente vinculado e os dados do paciente.
- Timeline de cuidados, com cada atendimento conforme acontece.

- Gráfico de sinais vitais, atualizado a cada novo registro.
- Filtro de histórico e perfil.

**Fluxo de uso:** a família recebe o convite, cria a conta, cadastra o paciente no onboarding e passa a acompanhar a timeline, os sinais vitais e o histórico completo.

## 4. Fluxos que atravessam os perfis

### 4.1 Entrada do paciente

O paciente entra pela família. O admin convida a família, que cria a conta e cadastra o paciente com os dados clínicos no onboarding. O admin então revisa, vincula e coloca o paciente na escala. Isso garante que nenhum paciente exista sem um responsável familiar ligado a ele.

### 4.2 Plantão do cuidador

Com a escala definida, o cuidador só consegue iniciar o plantão do paciente marcado para aquele dia. Durante o plantão, cada cuidado vira um registro. Se não houver internet, os registros ficam numa fila e sobem sozinhos depois. Ao final, a evolução resume o plantão para o próximo profissional.

### 4.3 Acompanhamento da família

Cada registro do cuidador aparece para a família em tempo real, respeitando o que é visível. A família vê a timeline do dia, o gráfico de sinais vitais e o histórico, sem acessar dados de outros pacientes da empresa.

## 5. Segurança e integridade dos dados

Como o app roda no aparelho do usuário, a segurança de verdade mora nas regras do Firebase, que continuam valendo mesmo que alguém tente acessar os dados por fora do app. O projeto adota negação por padrão, isolamento por empresa, acesso por perfil e registros clínicos que só podem ser criados, nunca editados ou apagados pelo aplicativo. Há ainda uma trilha de auditoria de login e logout. Os detalhes estão nos documentos de segurança e integridade que acompanham o projeto.

## 6. Investimento e infraestrutura

Esta seção responde uma dúvida comum: o app exige alguma infraestrutura paga? A resposta curta é que ele começa sem investimento de operação e passa a ter um investimento pequeno e proporcional ao uso conforme a base cresce. Não há servidor próprio nem licença de software.

### 6.1 Infraestrutura

A base de nuvem do app tem uma faixa de uso gratuita generosa, que atende bem o começo de uma operação pequena, com poucos pacientes e uma equipe enxuta. Conforme o número de pacientes e de famílias acompanhando ao mesmo tempo cresce, o app passa a um modelo por uso, com investimento proporcional e baixo para uma escala pequena ou média.

Vale uma nota de engenharia: como as fotos dos registros ficam guardadas dentro do banco, elas ocupam espaço mais rápido conforme o volume aumenta. No médio prazo, mover as fotos para o armazenamento de arquivos dedicado deixa o investimento de infraestrutura ainda mais enxuto. É uma otimização recomendada quando a base crescer.

### 6.2 Publicação nas lojas

Para colocar o app nas lojas, cada plataforma pede um registro de desenvolvedor. Segue o que envolve.

| Item                      | Quando         | Observação                                                |
|---------------------------|----------------|-----------------------------------------------------------|
| Apple Developer           | Anual          | Obrigatório para publicar e instalar em iPhone.           |
| Google Play               | Uma vez        | Registro único, sem renovação.                            |
| Build na nuvem (EAS)      | Incluído       | Plano gratuito atende à operação.                         |
| Infraestrutura (Firebase) | Conforme o uso | Gratuita no início, investimento proporcional ao crescer. |

### 6.3 Resumo

O investimento inicial se resume aos registros de desenvolvedor da Apple e da Google. A infraestrutura começa gratuita e, quando a base cresce, vira um investimento por uso, baixo e proporcional. Não há servidor próprio, licença de software nem mensalidade de plataforma de build. Em resumo, o investimento é enxuto, previsível e cresce junto com o número de clientes.

## 7. Considerações finais

- App Check, para garantir que as chamadas ao Firebase venham só do app verdadeiro.
- Notificações no celular, que dependem da conta paga da Apple, já prevista.
- Mover as fotos para o Firebase Storage quando o volume crescer, para enxugar o investimento de infraestrutura.
- Testes automatizados das regras de segurança no emulador do Firebase.